

Presentation of the NMPDE project

Stefano Mancini

September 3, 2024

Roadmap

- 1 Overview of the problem
- 2 Solving the ODE
- 3 Mesh adaptivity
- 4 Tests and results

Overview of the problem

We consider the heat equation:

$$\begin{cases} u_t(x, t) - \Delta u(x, t) = f(x, t) & (x, t) \in \Omega \times (t_0, t_1] \\ u(x, t_0) = u_0(x) & x \in \Omega \\ u(x, t) = g(x, t) & (x, t) \in \partial\Omega \times [t_0, t_1] \end{cases}$$

In particular, the domain Ω is a `hyper_L` (2D and 3D case) or the interval $[-1, 1]$ (1D case).

We have to address the following issues:

- How do we discretize the PDE?
- What weak form do we choose?

Overview of the problem

We consider the heat equation:

$$\begin{cases} u_t(x, t) - \Delta u(x, t) = f(x, t) & (x, t) \in \Omega \times (t_0, t_1] \\ u(x, t_0) = u_0(x) & x \in \Omega \\ u(x, t) = g(x, t) & (x, t) \in \partial\Omega \times [t_0, t_1] \end{cases}$$

In particular, the domain Ω is a `hyper_L` (2D and 3D case) or the interval $[-1, 1]$ (1D case).

We have to address the following issues:

- How do we discretize the PDE?
- What weak form do we choose?

Discretization

To solve the PDE numerically, we use semi-discretization. There are two ways to do this:

- Rothe's method, i.e. discretize in time first and get a PDE which is only space-dependent; then treat the resulting problem with a finite element method.
- The method of lines, i.e. discretize in space first and get an ODE that depends only on time; then solve the ODE with an IVP solver.

The method of lines saves us the effort of writing the time stepping by hand. Moreover, professional libraries for ODE solution, such SUNDIALS, already implement strategies for time step adaptivity.

Discretization

To solve the PDE numerically, we use semi-discretization. There are two ways to do this:

- Rothe's method, i.e. discretize in time first and get a PDE which is only space-dependent; then treat the resulting problem with a finite element method.
- The method of lines, i.e. discretize in space first and get an ODE that depends only on time; then solve the ODE with an IVP solver.

The method of lines saves us the effort of writing the time stepping by hand. Moreover, professional libraries for ODE solution, such SUNDIALS, already implement strategies for time step adaptivity.

Choosing the weak form

A standard way to solve stationary problems is to split the solution u into a sum $u_0 + u_g$, such that u_0 satisfies homogeneous boundary conditions and $u_g = g$ at the boundary (in terms of trace). Then we solve for $u_0 \in H_0^1(\Omega)$.

We can do the same for time dependent problems, but with the disadvantage that the boundary condition g changes over time. Thus, we use Nitsche's method and incorporate the boundary condition directly in the weak form.

Choosing the weak form

A standard way to solve stationary problems is to split the solution u into a sum $u_0 + u_g$, such that u_0 satisfies homogeneous boundary conditions and $u_g = g$ at the boundary (in terms of trace). Then we solve for $u_0 \in H_0^1(\Omega)$.

We can do the same for time dependent problems, but with the disadvantage that the boundary condition g changes over time. Thus, we use Nitsche's method and incorporate the boundary condition directly in the weak form.

Choosing the weak form

The weak form we obtain by using Nitsche's method is the following:

$$(u_t, v) + (\nabla u, \nabla v) - \langle \nabla u \cdot n, v \rangle - \langle u, \nabla v \cdot n \rangle + \gamma \langle u, v \rangle = \\ (f, v) - \langle g, \nabla v \cdot n \rangle + \gamma \langle g, v \rangle$$

where $(\cdot, \cdot)$ is the inner product in Ω , $\langle \cdot, \cdot \rangle$ the inner product in $\Gamma = \partial\Omega$ and n is the normal to Γ . The parameter $\gamma > 0$ is fixed.

Choosing the weak form

Let V_h be our FE space and let φ_i be the basis functions. Assume that the numerical solution $u_h \in V_h$ can be written as $u_h(x, t) = \sum U_i(t)\varphi_i(x)$. This leads to the following ODE:

$$M\mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u})$$

where

$$\begin{aligned} \mathbf{u}_i &= U_i(t), \quad M_{ij} = (\varphi_i, \varphi_j), \quad f_I(t, \mathbf{u}) = J\mathbf{u} \\ f_E(t, \mathbf{u})_i &= (f(\cdot, t), \varphi_i) + \gamma \langle g(\cdot, t), \varphi_i \rangle - \langle g(\cdot, t), \nabla \varphi_i \cdot \mathbf{n} \rangle \\ J_{ij} &= -(\nabla \varphi_i, \nabla \varphi_j) + \langle \varphi_i, \nabla \varphi_j \cdot \mathbf{n} \rangle + \langle \nabla \varphi_i \cdot \mathbf{n}, \varphi_j \rangle - \gamma \langle \varphi_i, \varphi_j \rangle \end{aligned}$$

The matrices M and J are independent of time!

ARK methods

Consider the IVP

$$\begin{cases} M(t)y' = f_E(t, y) + f_I(t, y), & t \in [t_0, T] \\ y(t_0) = y_0 \end{cases}$$

with $y \in \mathbb{R}^n$ and $M(t)$ nonsingular. If $\hat{f}_E(t, y) = M(t)^{-1} f_E(t, y)$ and $\hat{f}_I(t, y) = M(t)^{-1} f_I(t, y)$, an ARK method has the form:

$$z_i = y_{n-1} + h_n \sum_{j=1}^{i-1} A_{i,j}^E \hat{f}_E(t_{n,j}^E, z_j) + h_n \sum_{j=1}^i A_{i,j}^I \hat{f}_I(t_{n,j}^I, z_j), \quad i = 1, \dots, s,$$

$$y_n = y_{n-1} + h_n \sum_{i=1}^s \left(b_i^E \hat{f}_E(t_{n,i}^E, z_i) + b_i^I \hat{f}_I(t_{n,i}^I, z_i) \right),$$

$$\tilde{y}_n = y_{n-1} + h_n \sum_{i=1}^s \left(\tilde{b}_i^E \hat{f}_E(t_{n,i}^E, z_i) + \tilde{b}_i^I \hat{f}_I(t_{n,i}^I, z_i) \right).$$

ARK methods

We are treating the IVP as a mixed stiff/nonstiff problem:

- f_E contains the "slow" (*nonstiff*) time scale components of the system. This part will be integrated using explicit methods.
- f_I contains the "fast" (*stiff*) time scale components of the system. This will be integrated using implicit methods.
- We are coupling two s -stage embedded methods: an explicit one and a diagonally implicit one.

ARK methods

When we use ARK methods, an implicit system of the form

$$G(z_i) := Mz_i - h_n A'_{i,i} f_l(t'_{n,i}, z_i) - a_i = 0$$

must be solved for each stage z_i , with $i = 1, \dots, s$, where we have the data

$$a_i := My_{n-1} + h_n \sum_{j=1}^{i-1} [A^E_{i,j} f_E(t^E_{n,j}, z_j) + A^l_{i,j} f_l(t^l_{n,j}, z_j)].$$

In our specific case, $f_l(t, y)$ will depend linearly on y .

ARK methods

The default solution strategy is a variant of Newton's method:

$$z_i^{m+1} = z_i^m + \delta^{m+1}.$$

Here δ^{m+1} solves the linear Newton system

$$N(z_i^m)\delta^{m+1} = -G(z_i^m),$$

where

$$N := M - \gamma J, \quad J := \frac{\partial f_l}{\partial y}, \quad \gamma := h_n A'_{i,i}.$$

Since $f_l(t, y)$ depends linearly on y , each system will be solved using a single Newton iteration. The initial guess will be a predicted value $z_i(0)$ that is computed explicitly from the previously-computed data.

Final remarks

Let us write again the IVP that we will try to solve:

$$\begin{cases} M\mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u}), & t \in [t_0, t_1] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

where $\mathbf{u}_0$ is the initial condition $u_0(x)$, interpolated with respect to the basis functions φ_i .

In particular, the linear systems involved in this specific IVP have symmetric and positive definite matrices, hence we can use as a solver the CG method with an AMG preconditioner.

Time step size and minimal mesh size

- For stationary problems, the general approach is "make the mesh as fine as it is necessary".
- For time dependent problems, we typically have error estimates of the form

$$\|e\| \leq O(k^p + h^q)$$

where p , q are the convergence orders of the time and space discretization, respectively.

- We can only make the error small if we decrease both terms. Ideally, this suggests to choose $k \propto h_{min}^{q/p}$, because at least for problems with non-smooth solutions, the error is typically localized in the cells with the smallest mesh size.

Time step size and minimal mesh size

- Problem: a significantly smaller time step means having to solve the global linear system more often. This implies a non negligible growth of the computational effort.
- Therefore, we avoid to make the local mesh size arbitrarily small. Rather, we set a maximal level of refinement and, when we flag cells for refinement, we simply do not refine those cells whose children would exceed this maximal level of refinement.
- We enforce a minimal mesh refinement level as well, in order to be ready for a sudden change in the solution.

Test functions from different meshes

Let us recall the IVP we are solving:

$$\begin{cases} M\mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u}), & t \in [t_0, t_1] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

Suppose that at some time $t_* > t_0$ we decide to refine the mesh.

This leads to a new set of basis functions $\{\hat{\varphi}_i\}$, which in turn leads to new matrices $\hat{M}$ and $\hat{J}$, new functions $\hat{f}_E$ and $\hat{f}_I$.

Moreover, the solution, starting from $t = t_*$, will have to be expressed as a linear combinations of the new basis functions:

$$u(x, t) \approx \sum \hat{U}_i(t) \hat{\varphi}_i(x).$$

Test functions from different meshes

Let us recall the IVP we are solving:

$$\begin{cases} M\mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u}), & t \in [t_0, t_1] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

Suppose that at some time $t_* > t_0$ we decide to refine the mesh. This leads to a new set of basis functions $\{\hat{\varphi}_i\}$, which in turn leads to new matrices $\hat{M}$ and $\hat{J}$, new functions $\hat{f}_E$ and $\hat{f}_I$. Moreover, the solution, starting from $t = t_*$, will have to be expressed as a linear combinations of the new basis functions:

$$u(x, t) \approx \sum \hat{U}_i(t) \hat{\varphi}_i(x).$$

Test functions from different meshes

We thus obtain a new IVP:

$$\begin{cases} \hat{M}\hat{\mathbf{u}}' = \hat{f}_E(t, \hat{\mathbf{u}}) + \hat{f}_I(t, \hat{\mathbf{u}}), & t \in [t_*, t_1] \\ \hat{\mathbf{u}}(t_*) = \hat{\mathbf{u}}_* \end{cases}$$

where $\hat{\mathbf{u}}_i = \hat{U}_i$.

We get the vector $\hat{\mathbf{u}}_*$ by *transferring* the approximated solution at time t_* from the old mesh to the new one:

$$\hat{\mathbf{u}}_* = I_{t_*} \mathbf{u}(t_*).$$

I_t is the interpolation operator onto the finite element space used at time t .

Test functions from different meshes

We thus obtain a new IVP:

$$\begin{cases} \hat{M}\hat{\mathbf{u}}' = \hat{f}_E(t, \hat{\mathbf{u}}) + \hat{f}_I(t, \hat{\mathbf{u}}), & t \in [t_*, t_1] \\ \hat{\mathbf{u}}(t_*) = \hat{\mathbf{u}}_* \end{cases}$$

where $\hat{\mathbf{u}}_i = \hat{U}_i$.

We get the vector $\hat{\mathbf{u}}_*$ by *transferring* the approximated solution at time t_* from the old mesh to the new one:

$$\hat{\mathbf{u}}_* = I_{t_*} \mathbf{u}(t_*).$$

I_t is the interpolation operator onto the finite element space used at time t .

A posteriori error estimation

To decide whether a mesh refinement is needed, we use the Kelly error estimator:

$$\eta_T := \sum_{F \subset \partial T} \frac{1}{2} h_F^{\frac{1}{2}} \| [\![\nabla u_h]\!] \|_{0,F}.$$

Here u_h represents the approximated solution. We fix some checkpoints $t^{(1)}, t^{(2)}, \dots \in [t_0, t_1]$ and refine the mesh at time $t^{(k)}$ if

$$\sum_{T \in \mathcal{T}_h} \eta_T^2 > \text{tol}$$

for a fixed tolerance `tol`.

We leave room for doing multiple refinements at the same time, being careful in order to avoid a never ending refinement loop.

Some remarks on the program

- The customizable parameters are collected in a .prm file and are handled internally via a `ParameterHandler` object.
- The assembly of matrices and vectors is done via a `MeshWorker` loop.
- The AMG preconditioner is implemented in `deal.II` with the `TrilinosWrappers::PreconditionAMG` class. Trilinos matrices and vectors are used as well, but this is not mandatory.
- As default, the program checks if the mesh has to be refined at each output step. However, additional checkpoints can be added uniformly between consecutive outputs.

Tests with no adaptive refinement

Choosing the "right" value of the Nitsche parameter γ is not straightforward:

- If it is too low, then the method becomes unstable.
- Raising γ leads to an increase of computational cost, because ARKode will use a significantly lower timestep size.
- Increasing γ until a certain trade-off value guarantees a gain in accuracy. Going beyond that is useless.
- A higher mesh refinement or FE degree do require a higher value of γ to maintain the stability of the method.

Tests with no adaptive refinement

Relation between timestep size and mesh size:

- Increasing the refinement of the mesh leads to a better accuracy, at the cost of more ARKode steps.
- Increasing the FE degree leads also to a better accuracy, *sometimes* with less cost in terms of ARKode steps than increasing the global mesh refinement.
- A limit of our program: in general, ARKode adapts the time step automatically to fulfill a desired tolerance, which has no relation with the mesh size at all. Hence the FEM error or the ARK error may likely prevail on the other, wasting computational resources.
- A possible consequence of this: sometimes the L^2 error becomes oscillatory throughout time, even though a certain minimal accuracy is maintained.

Tests with no adaptive refinement - 1D

$$u(x, t) = 2 - (x - t)^2, \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	0.0170196	253	0.0103807
$\gamma = 20$	0.0132822	534	0.00485258
$\gamma = 50$	0.0139157	1147	0.000898033
<code>in_ref = 5</code> , $\gamma = 10$	explodes		
<code>in_ref = 5</code> , $\gamma = 20$	0.00312233	1410	0.0009152
<code>in_ref = 5</code> , $\gamma = 50$	0.000916882	2187	0.00141454
<code>in_ref = 5</code> , $\gamma = 100$	0.000900294	4605	0.000753134
<code>fe_deg = 2</code> , $\gamma = 10$	explodes		
<code>fe_deg = 2</code> , $\gamma = 20$	9.34488×10^{-7}	680	0.000888443
<code>fe_deg = 2</code> , $\gamma = 50$	1.34329×10^{-5}	1773	0.00235106

Tests with no adaptive refinement - 1D

$$u(x, t) = 2 - 10(x - \cos(\pi t))^2, \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	0.170172	297	0.0132492
<code>in_ref</code> = 5, γ = 100	0.0090014	4801	0.00107145
<code>fe_deg</code> = 2, γ = 50	3.19038×10^{-6}	1953	0.00164072
<code>fe_deg</code> = 2, γ = 100	0.000306053	3747	0.00472022

Steeper solutions require finer meshes or higher FE degree.

Tests with no adaptive refinement - 1D

$$u(x, t) = 4t(x^2 - x), \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	0.123128	209	0.0143597
in_ref = 5, $\gamma = 100$	0.00598402	3066	0.00139227
fe_deg = 2, $\gamma = 50$	7.25037×10^{-6}	1326	0.000842335
fe_deg = 2, $\gamma = 100$	1.82201×10^{-5}	2388	0.00048086

Steeper solutions require finer meshes or higher FE degree (again).

Tests with no adaptive refinement - 2D

$$u(x, y, t) = 2 - (x - t)^2, \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	0.00231121	2277	0.00067853
<code>in_ref</code> = 4, γ = 50	0.000641402	11437	0.000202871
<code>fe_deg</code> = 2, γ = 100	3.80978×10^{-5}	11559	0.000208635

Tests with no adaptive refinement - 2D

$$u(x, y, t) = 2 - 10(x - \cos(\pi t))^2, \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	0.0231104	4403	0.00292041
<code>in_ref</code> = 4	0.00612146	9044	0.00161523
<code>in_ref</code> = 4, $\gamma = 20$	0.00631621	16450	0.00100964
<code>fe_deg</code> = 2, $\gamma = 20$	5.88644×10^{-5}	8937	0.00179902

Tests with `fe_degree` = 2 and $\gamma = 50, 100$ were aborted because of the too large amount of work for ARKode: even though the integration continued, the time step was frequently around 10^{-5} , and sometimes errors from ARKode showed up.

Tests with no adaptive refinement - 2D

$$u(x, y, t) = \exp\left[-2\left(\left(x - \frac{1}{2}\cos\left(\pi\left(t + \frac{1}{2}\right)\right)\right)^2 + \left(y - \frac{1}{2}\sin\left(\pi\left(t + \frac{1}{2}\right)\right)\right)^2\right)\right]$$

with $t \in \left[0, \frac{3}{2}\right]$.

	Error at t_1	ARK steps	Last stepsize
Default settings	0.00276206	2055	0.000907226
<code>in_ref = 4</code>	0.000723828	3167	0.000997511
<code>in_ref = 4, $\gamma = 20$</code>	0.000738421	6041	0.000434012

Tests with no adaptive refinement - 3D

$$u(x, y, z, t) = 2 - (x - t)^2, \text{ with } t \in [-1, 1].$$

	Error at t_1	ARK steps	Last stepsize
Default settings	program crash		
$\gamma < 50$	program crash		
$\gamma = 100$	0.00307315	4585	0.000431185
<code>in_ref</code> = 2	0.00789524	631	0.00130313

Generally speaking, compared to the 2D case, 3D problems require a higher value of γ for the method to work well. In particular, there seems to be a bug in deal.II, because choosing γ too low leads to an unexpected crash.

Some words on adaptive refinement

- Adaptive mesh refinement encounters some troubles in 1D, because we cannot talk about a "face diameter" in dimension 1. Hence, we may have to increase γ slightly (a priori) to prevent the numerical solution to explode.
- Tweaking the parameters, in particular the tolerance for the Kelly error estimator and the refinement fractions, is important to get a good result.
 - When a solution tends to return to a flat state, we would like to have a coarser mesh. Hence we have to increase the refinement bottom fraction. Unfortunately, this clashes a bit with requiring the mesh to be finer when necessary.
 - If the refinement bottom fraction is too low, the number of cells tends to grow with time. This may lead to using more degrees of freedom than actually needed.